



Overview Timing Analysis

PRESENTED BY

Balajothi K

– 12300635

Introduction to Timing Analysis

What is Timing Analysis?

Timing analysis is the process of verifying that all signals in a digital circuit arrive at the right time to ensure correct operation.

It checks whether data launched by one flip-flop reaches the next flip-flop within the clock period, meeting setup and hold timing requirements.

Dynamic (Simulation-Based) Timing Analysis :

This is what we do in Verilog simulation tools like ModelSim, Vivado, or Cadence NC-Sim. We define delays manually in the code or testbench — the simulator runs over time and shows signal transitions.

ex.

```
always #5 clk = ~clk; // Clock with 10 ps period
#25 a = 1;           // Apply input after 25 ps
```

Static Timing Analysis (STA) :

Unlike simulation, STA doesn't run any testbench or #delay statements.

It mathematically checks all timing paths in the circuit — from inputs to outputs — using:

The netlist (.v), The technology library (.lib), The timing constraints (.sdc),
It calculates setup and hold slacks for every path, ensuring the design meets timing for all possible input conditions.

STA	Simulation
No test-vector required	Test vector required
No check of functionality	Checks functionality
Analysis performed taking pessimistic view of delays and other attributes of the design	Simulation done based on specified test vectors and delays

Key Timing Parameters

1. Arrival Time (AT):

$$T_{arrival}^{max} = T_{launch} + T_{comb,max}$$

→ Time at which data arrives at the capture flip-flop input through the longest combinational path.

→ Depends on clock-to-Q delay and max propagation delay of logic between flops.

2. Required Time (RT):

$$T_{required}^{setup} = T_{clock} - T_{setup}$$

→ The latest time by which data must arrive at the capture flip-flop to meet setup.

→ Determined by clock period and setup time of the capturing register.

3. Setup Time (T_setup):

→ Minimum time before the clock edge during which input D must remain stable.

→ Ensures correct sampling of data at the clock edge.

4. Hold Time (T_hold):

→ Minimum time after the clock edge during which input D must remain stable.

→ Prevents incorrect data latching due to early arrival.

3. Setup Slack (S_setup):

$$Slack_{setup} = T_{required}^{setup} - T_{arrival}^{max}$$

→ Positive slack means timing met; negative slack means setup violation.

→ Calculated for the maximum delay path (worst-case slow corner).

3. Hold Slack (H_hold):

$$Slack_{hold} = T_{arrival}^{min} - T_{required}^{hold}$$

→ Ensures data remains stable for hold time after the clock edge.

→ Calculated for minimum delay path (fastest signal arrival).

Clock-to-Q Delay (T_{clk→Q}):

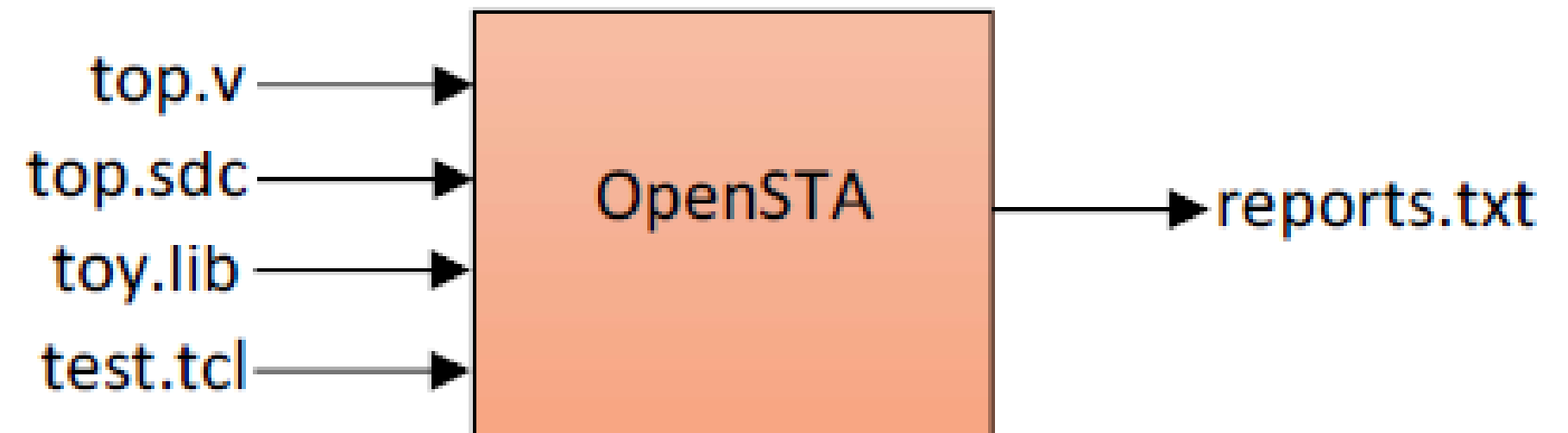
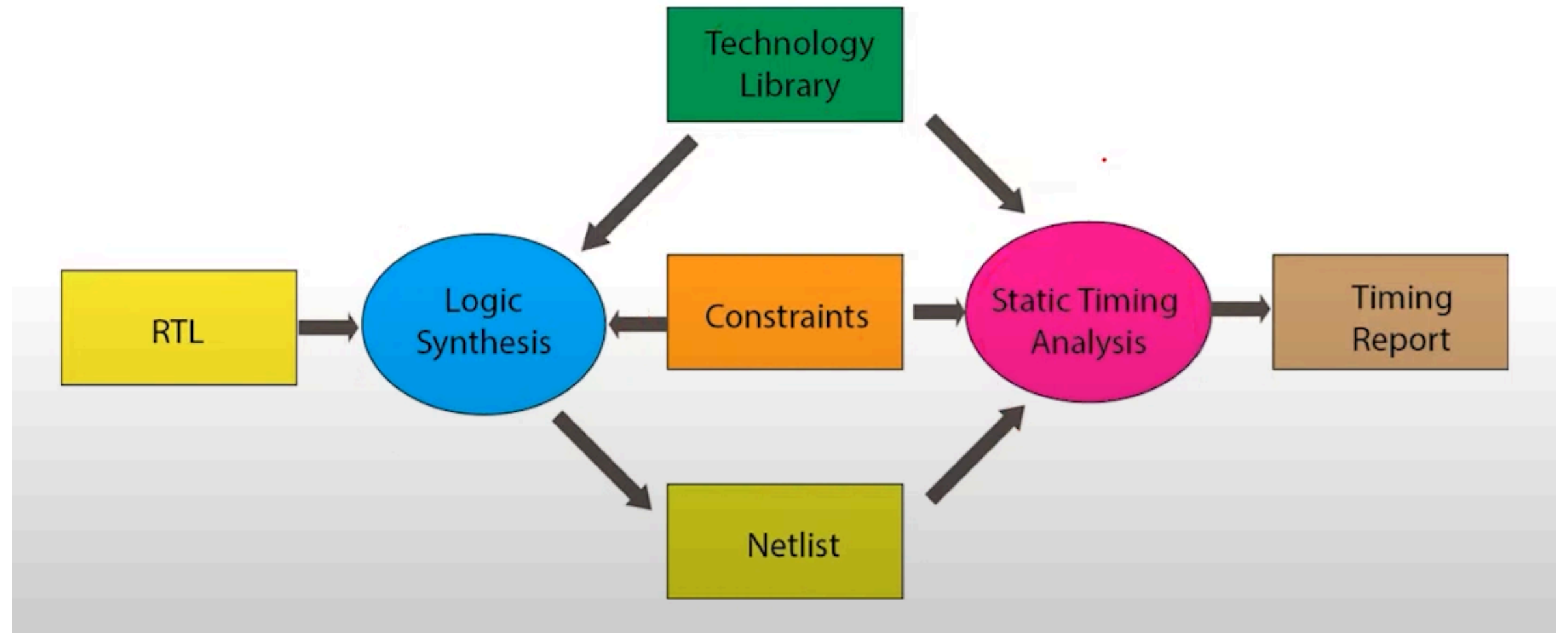
$$T_{clk→Q} = T_{output} - T_{clock_edge}$$

Timing Analysis Flow

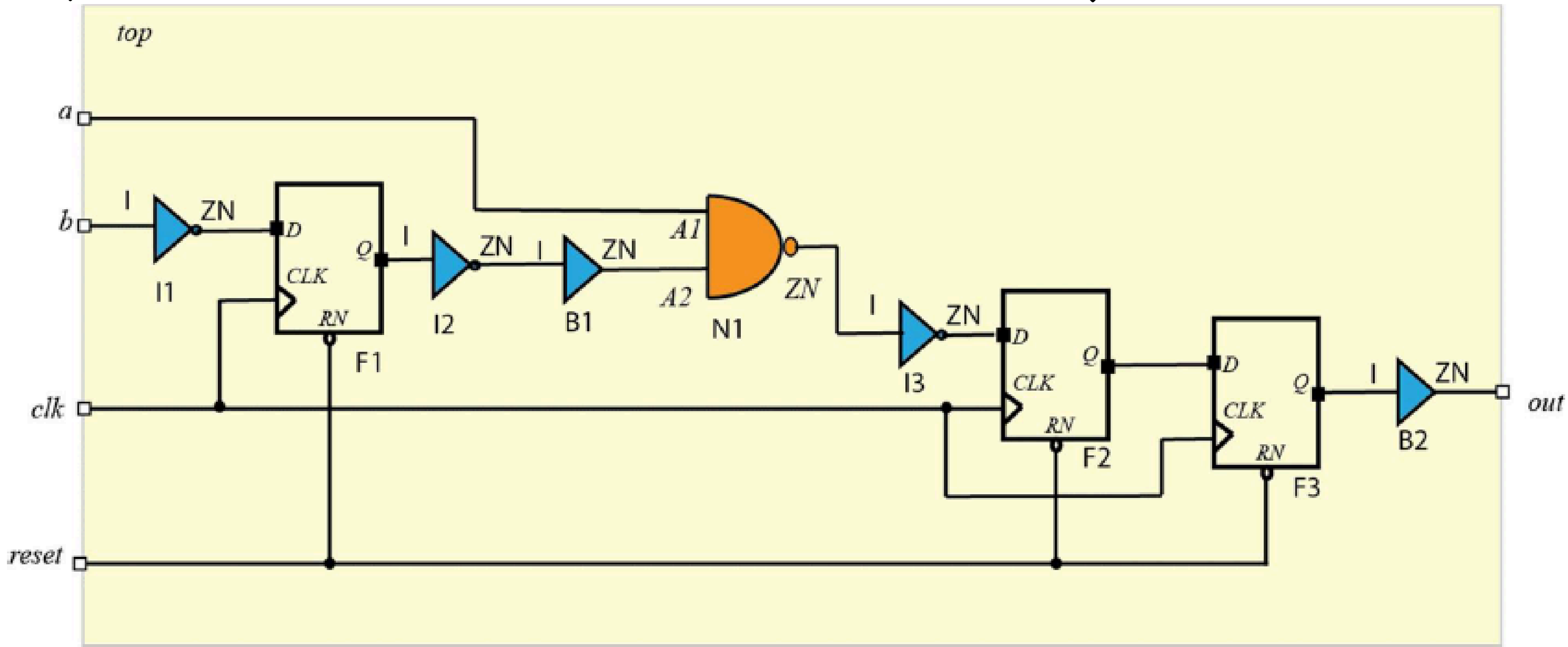
The timing analysis process begins with the RTL design written in Verilog, which is synthesized into a gate-level netlist using a logic synthesis tool. Design constraints such as clock definitions and input/output delays are provided along with a technology library that defines cell timing characteristics.

These inputs — netlist, constraints, and library — are analyzed using a Static Timing Analysis (STA) tool like OpenSTA to evaluate setup and hold times.

Finally, a detailed timing report is generated to verify whether the design meets timing requirements.



Manual Problem Example



In the sequential circuit shown below, the clock period is 1000 ps. Each INV = 100 ps, BUF = 100 ps, NAND2 = 120 ps, Clock-to-Q = 50 ps, Setup time = 40 ps, and Hold time = 10 ps. Ignoring wire delays and clock skew, find the Setup Slack and Hold Slack at node F2/D ?

OpenSTA Demo

```
balas_byter@ByteBuddy: /mn  × + ▾ - □ ×
balas_byter@ByteBuddy:~$ cd /mnt/c/Users/kbala/OpenSTA_Pro
ject
balas_byter@ByteBuddy:/mnt/c/Users/kbala/OpenSTA_Project$
ls
test.tcl top.sdc top.v toy.lib
balas_byter@ByteBuddy:/mnt/c/Users/kbala/OpenSTA_Project$
balas_byter@ByteBuddy:/mnt/c/Users/kbala/OpenSTA_Probababa
balas_byter@ByteBuddy:/mnt/c/Users/kbala/OpenSTA_Project$
gedit top.v gedit test.tcl gedit top.sdc
```

```
Q | 📁 | ⬅️ | ➡️ | ✂️ | 📄 | 📁 | ✖️ | // | 📊 | 💡 |
1 module top(a, b, clk, reset, out);
2   input a, b, clk, reset;
3   output out;
4   wire y1,y2,y3,y4,y5,y6,y7,y8;
5   INV I1 ( .I(b), .ZN(y1));
6   DFFRNQ F1 ( .CLK(clk), .D(y1), .Q(y2), .RN(reset));
7   INV I2 ( .I(y2), .ZN(y3));
8   BUF B1 ( .I(y3), .Z(y4));
9   NAND2 N1( .A1(a), .A2(y4), .ZN(y5));
10  INV I3 ( .I(y5), .ZN(y6));
11  DFFRNQ F2 ( .CLK(clk), .D(y6), .Q(y7), .RN(reset));
12  DFFRNQ F3 ( .CLK(clk), .D(y7), .Q(y8), .RN(reset));
13  BUF B2 ( .I(y8), .Z(out));
14 endmodule
15
```

```
1|read_liberty toy.lib
2 read_verilog top.v
3 link_design top
4 read_sdc top.sdc
```

```
1 create_clock -name CLK -period 1000 [get_ports clk]
2 set_input_delay 5 -clock CLK [get_ports a]
3 set_input_delay 5 -clock CLK [get_ports b]
4 set_output_delay 5 -clock CLK [get_ports out]
```


Results Overview

```
balas_byter@ByteBuddy:~$ cd /mnt/c/Users/kbala/OpenSTA_Project
balas_byter@ByteBuddy:/mnt/c/Users/kbala/OpenSTA_Project$ ls
test.tcl top.sdc top.v toy.lib
balas_byter@ByteBuddy:/mnt/c/Users/kbala/OpenSTA_Project$ sta
OpenSTA 2.7.0 585ff0c98b Copyright (c) 2025, Parallax Software, Inc.
License GPLv3: GNU GPL version 3 <http://gnu.org/licenses/gpl.html>
```

This is free software, and you are free to change and redistribute it under certain conditions; type 'show_copyright' for details.
This program comes with ABSOLUTELY NO WARRANTY; for details type 'show_warranty'.

```
% source test.tcl
Startpoint: F1 (rising edge-triggered flip-flop clocked by CLK)
Endpoint: F2 (rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max
```

Delay	Time	Description
0.00	0.00	clock CLK (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	^ F1/CLK (DFFRNQ)
5.66	5.66	v F1/Q (DFFRNQ)
1.63	7.29	^ I2/ZN (INV)
5.20	12.49	^ B1/Z (BUF)
1.91	14.40	v N1/ZN (NAND2)
1.85	16.25	^ I3/ZN (INV)
0.00	16.25	^ F2/D (DFFRNQ)
	16.25	data arrival time
1000.00	1000.00	clock CLK (rise edge)
0.00	1000.00	clock network delay (ideal)
0.00	1000.00	clock reconvergence pessimism
	1000.00	^ F2/CLK (DFFRNQ)
-8.36	991.64	library setup time
	991.64	data required time

	991.64	data required time
	-16.25	data arrival time

	975.39	slack (MET)

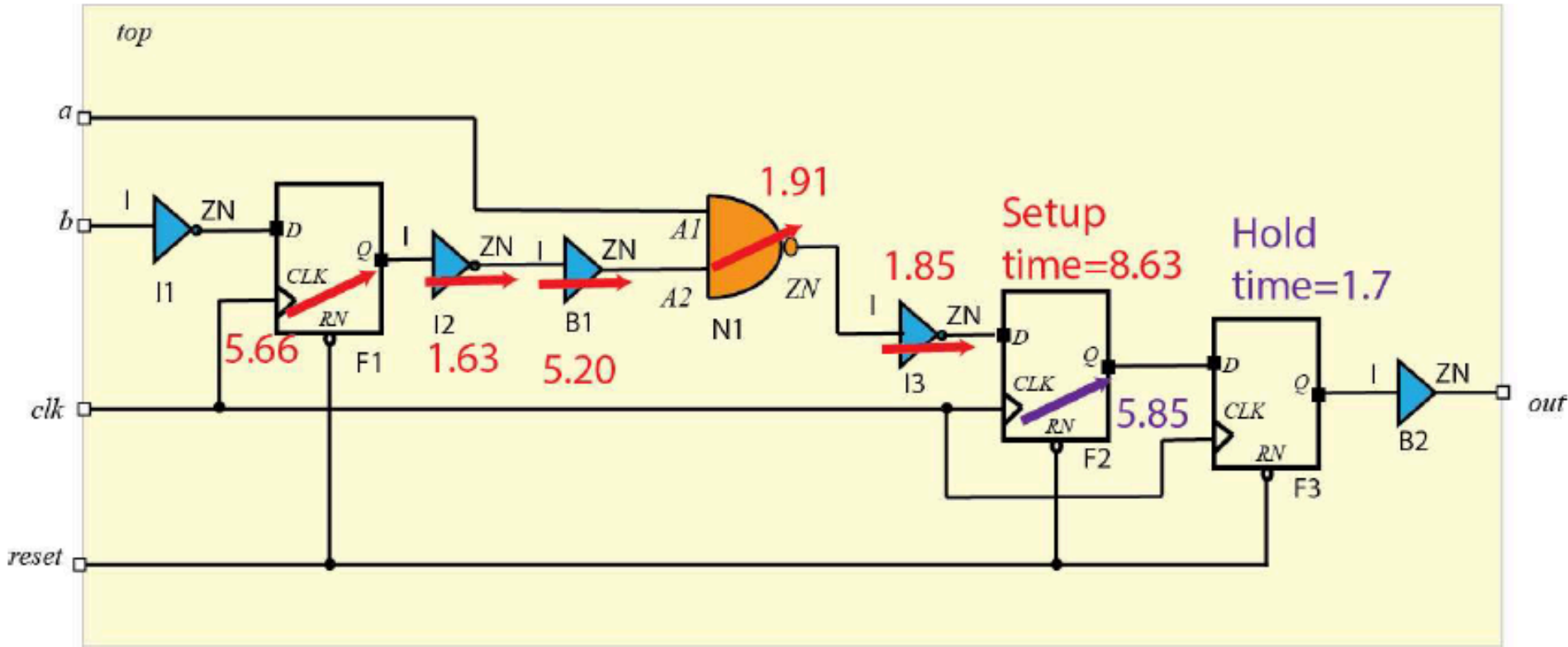
```
Startpoint: F2 (rising edge-triggered flip-flop clocked by CLK)
Endpoint: F3 (rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: min
```

Delay	Time	Description
0.00	0.00	clock CLK (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	^ F2/CLK (DFFRNQ)
5.85	5.85	^ F2/Q (DFFRNQ)
0.00	5.85	^ F3/D (DFFRNQ)
	5.85	data arrival time
0.00	0.00	clock CLK (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
	0.00	^ F3/CLK (DFFRNQ)
1.70	1.70	library hold time
	1.70	data required time

	1.70	data required time
	-5.85	data arrival time

	4.15	slack (MET)

%



Conclusion & Future Work

The figure illustrates the timing optimization loop in digital design. After performing Static Timing Analysis (STA) using the mapped netlist, standard cell libraries, and SDC constraints:

- If the timing target is met, the design proceeds to area recovery to reduce chip area and power.
- If timing violations exist, STA identifies the critical path and applies timing-driven transformations (such as gate sizing, buffering, or restructuring).
- Incremental STA is then run again to validate improvements — forming a continuous loop until timing closure is achieved.

Future efforts will focus on:

- Power and timing co-analysis for better performance-efficiency trade-offs.
- Multi-corner, multi-mode (MCM) timing optimization for reliable results across all conditions.
- Integration with synthesis tools to achieve faster and more accurate design convergence in modern VLSI flows.

